

# Glitch Hunter — Tech Stack

Sep 24, 2026 · shared by @Tanmay Singh

## What this is

Glitch Hunter is a PPO reinforcement-learning agent that plays a Super Mario Bros clone and is watched live through a web dashboard; a second pipeline turns anything it catches breaking the game's own rules into an evidence bundle (screenshot, GIF, Markdown/PDF report). The table below lists every technology in the project and exactly which part of it uses that technology.

## Tech Stack

| Layer                  | Technology              | Version  | Where it's used                                                                                                      |
|------------------------|-------------------------|----------|----------------------------------------------------------------------------------------------------------------------|
| Language               | Python                  | 3.12     | The entire codebase                                                                                                  |
| ML framework           | PyTorch                 | 2.5.1    | Neural network behind the PPO agent (loaded via Stable-Baselines3)                                                   |
| RL algorithm           | Stable-Baselines3 (PPO) | 2.9.0    | Trains and runs the agent ( <code>train_agent.py</code> , <code>dashboard_backend.py</code> )                        |
| Env interface          | Gymnasium               | 1.3.0    | Wraps the Mario game as a standard RL environment ( <code>custom_mario_env.py</code> )                               |
| Game engine            | Pygame                  | 2.6.1    | Renders the Mario clone itself ( <code>mario_clean/</code> , <code>mario_bugged/</code> ) and the observation frames |
| Numerical arrays       | NumPy                   | 2.5.2    | Coverage bitmaps, masks, observation tensors ( <code>exploration/</code> )                                           |
| Image/video processing | OpenCV (opencv-python)  | 5.0.0.93 | Resizes and JPEG-encodes each frame for the live video stream                                                        |

| Layer                         | Technology                      | Version       | Where it's used                                                                                      |
|-------------------------------|---------------------------------|---------------|------------------------------------------------------------------------------------------------------|
| Serialization                 | cloudpickle                     | 3.1.2         | Saves/loads trained checkpoints alongside Stable-Baselines3                                          |
| Web server                    | Flask                           | 3.1.3         | Serves the dashboard page and its REST API ( <code>app.py</code> )                                   |
| Real-time updates             | Flask-SocketIO                  | 5.6.1         | Pushes video frames, logs and incident alerts to the browser live                                    |
| Frontend                      | HTML / CSS / vanilla JavaScript | —             | The dashboard page itself ( <code>templates/</code> , <code>static/</code> ) — no frontend framework |
| Real-time client              | Socket.IO JS client             | vendored      | Browser side of the live stream ( <code>static/vendor/</code> , kept local for offline use)          |
| Incident reports — PDF        | fpdf2                           | 2.8.8         | Generates the PDF bug report per incident ( <code>reporting/render.py</code> )                       |
| Incident reports — images/GIF | Pillow                          | 12.3.0        | Builds the GIF and embeds screenshots in reports                                                     |
| Testing                       | pytest + pytest-cov             | 9.1.1 / 7.1.0 | The full automated test suite and its coverage gate ( <code>tests/</code> )                          |
| Linting                       | Ruff                            | 0.16.6        | Code style and static checks ( <code>tools/check.py</code> , CI)                                     |
| Type checking                 | mypy                            | 2.3.1         | Static type checks on Windows and Linux ( <code>tools/check.py</code> , CI)                          |
| Git hooks                     | pre-commit                      | 4.6.2         | Runs checks automatically before each commit                                                         |
| Package manager               | uv                              | —             | One-command install/dependency resolution ( <code>pyproject.toml</code> )                            |
| CI                            | GitHub Actions                  | —             | Runs lint, types, tests and artifact checks on every push ( <code>.github/workflows/ci.yml</code> )  |
| Training graphs (optional)    | TensorBoard                     | ≥ 2.18        | Optional training-curve viewer; not needed to watch the agent play                                   |

## Notes

- **torch is pinned to 2.5.1** on purpose: it's the newest version with a CUDA 12.1 build, which Stable-Baselines3 2.9.0 doesn't officially require but still works with. Installing SB3 normally would silently replace the GPU build with a CPU-only one — see `README.md`.
- **eventlet must never be installed.** Flask-SocketIO picks its async mode from whatever's in the environment, so the dashboard runs in plain threading mode deliberately — adding eventlet would switch that silently.
- **uv sync** is the one-command install and is the recommended path; the 3-step pip install in `requirements.txt` is the manual fallback and must be run in that exact order.